

Efficient FPGA Implementation of the Delayed LMS Algorithm

Lakshmi N

Dept. of Electronics & Communication
Engineering
Govt. Engineering College
Thrissur, India
neithilathlakshmi@gmail.com

Megha J Das

Dept. of Electronics & Communication
Engineering
Govt. Engineering College
Thrissur, India
meghajdas28@gmail.com

Job Chunkath 

Dept. of Electronics & Communication
Engineering
Govt. Engineering College
Thrissur, India
jobchunkath@gmail.com
<https://orcid.org/0000-0001-5106-0693>

Abstract—This paper presents the design and implementation of an adaptive filter utilizing the Delayed Least Mean Square (D-LMS) algorithm, optimized for hardware efficiency using Verilog HDL. The D-LMS algorithm improves upon the conventional Least Mean Square (LMS) algorithm by introducing a delay in the weight update process, which decouples error calculation and weight update steps. This modification significantly reduces the critical path, resulting in faster convergence and reduced computational complexity. Key components of the design, such as block carry look-ahead adders and Wallace tree multipliers, were optimized to enhance performance and minimize resource usage. The design was implemented on an FPGA platform, and simulation results show improvements in convergence speed, power consumption, and resource utilization compared to traditional LMS-based adaptive filters. This study demonstrates the practical advantages of the D-LMS algorithm in real-time signal processing applications, including noise cancellation, communication systems, and biomedical signal enhancement.

Keywords—Adaptive Filters, Delayed Least Mean Square (D-LMS) Algorithm, Field-Programmable Gate Arrays (FPGAs), Verilog HDL, Convergence Speed, LUTs, Real-Time Signal Processing

I. INTRODUCTION

Adaptive filtering is a fundamental technique in signal processing, with applications ranging from noise cancellation and system identification to channel equalization and biomedical signal enhancement. Among the numerous algorithms available for adaptive filtering, the Least Mean Square (LMS) algorithm is one of the most widely used due to its simplicity and ease of implementation. However, conventional LMS filters face significant challenges, including slow convergence, sensitivity to noise, and high computational complexity.

To address these limitations, the Delayed Least Mean Square (D-LMS) algorithm introduces a novel approach that decouples the error calculation and weight update processes by incorporating a delay. This separation optimizes the algorithm for hardware implementations, significantly reducing the critical path and improving convergence speed. The D-LMS algorithm has shown particular promise in real-time systems, where high-speed processing and efficient resource utilization are critical.

This paper explores the development and implementation of an adaptive filter based on the D-LMS algorithm using Verilog Hardware Description Language (HDL). The study focuses on optimizing the hardware design by employing components such as block carry look-ahead adders and

Wallace tree multipliers, which are essential for improving both performance and power efficiency. Simulation results demonstrate the advantages of the D-LMS filter over traditional LMS-based systems, highlighting its potential in various real-time signal processing applications.

II. LITERATURE SURVEY

Adaptive filtering is an essential technique in digital signal processing, with broad applications in areas such as noise cancellation, system identification, and communication systems. The Least Mean Square (LMS) algorithm, introduced by Widrow and Hoff [1], remains one of the most popular methods for adaptive filtering due to its simplicity and ease of implementation. However, conventional LMS filters exhibit certain limitations such as slow convergence and sensitivity to step-size selection, making them less suitable for high-speed real-time applications. To address these issues, several variations and enhancements to the LMS algorithm have been proposed in the literature.

A. Unified Theory of Adaptive Filters

Husøy and Abadi [2] present a comprehensive framework for adaptive filters, showing that many well-known adaptive filter algorithms can be viewed as special cases of this generalized theory. Their approach involves the use of a preconditioned Wiener-Hopf equation, iterative solution methods like Richardson iteration, and estimation strategies for the autocorrelation matrix. This framework not only provides theoretical insights but also allows for the derivation of performance metrics such as the learning curve and mean square error, which align well with simulations.

B. Critical-Path Analysis in LMS Adaptive Filters

Meher and Park [5] conducted an in-depth analysis of the critical paths in LMS filters, identifying ways to optimize architectures for high-speed, low-complexity implementations. The authors propose three distinct LMS filter designs, each offering trade-offs in terms of area, power efficiency, and maximum update frequency (MUF). The designs range from basic structures with minimal area and power consumption to more complex architectures that incorporate adaptation delays to increase MUF. These findings suggest that pipelining in LMS filters, while typically unnecessary, can significantly improve performance in specific high-speed applications.

C. Power-Efficient LMS Adaptive Filters

In another study, Meher and Park [6] introduce an area-delay-power efficient fixed-point LMS adaptive filter with reduced adaptation delay. The paper demonstrates how a novel partial product generator and optimized pipelining strategy result in lower adaptation delays and better area-delay-power efficiency. The fixed-point implementation of the LMS filter also offers significant improvements in power efficiency, with reductions of 17% in area-delay product (ADP) and 14% in energy-delay product (EDP) compared to existing designs. The paper also explores bit-level pruning techniques, which further reduce ADP and EDP by up to 20% and 9%, respectively, without significantly impacting the steady-state error.

D. Parallel Processing in LMS Algorithms

Dong, Li, and Wang [4] investigated the parallel processing capabilities of the Delayed LMS (D-LMS) algorithm through an FPGA-based implementation. Their study highlights the advantages of integrating parallelism with the D-LMS approach, which improves data throughput and accelerates convergence compared to traditional LMS algorithms. The Parallel Delayed LMS (PDLMS) algorithm discussed in this paper also allows for enhanced resource utilization, making it suitable for real-time systems that demand high-speed processing.

E. High-Speed Multiplication Techniques

The efficiency of hardware implementations of adaptive filters is often determined by the underlying arithmetic operations. Baugh and Wooley [3] presented a high-speed multiplication algorithm based on two's complement parallel array multiplication, which optimizes the generation of partial products through AND operations between bits of the multiplier and multiplicand. Their method transforms the problem of multiplication into a series of parallel additions, which significantly reduces the time required for each multiplication operation. This method is particularly beneficial in applications such as the D-LMS algorithm, where multiplication is a frequent operation.

F. Application of the Delayed LMS Algorithm

The Delayed LMS (D-LMS) algorithm, which forms the core of this study, addresses many of the inherent challenges in the standard LMS algorithm, such as slow convergence and high computational complexity.

The D-LMS algorithm decouples the error calculation and weight update processes, introducing a delay that allows for more efficient processing and faster clock cycles [7]. The delayed weight update results in improved tracking performance, reduced steady-state error, and enhanced stability, making it an attractive option for real-time adaptive filtering applications.

III. SYSTEM DESIGN

The implementation of the concept centers around the design, development, and hardware implementation of an adaptive filter using the Delayed Least Mean Square (D-LMS) algorithm. The key objective is to improve the performance of the traditional LMS algorithm by introducing a delay in the weight update process, which enhances convergence speed and reduces computational complexity. The methodology can be broken down into the following parts.

A. Overview of LMS and D-LMS Algorithms

The first part of the design involves a review of the Least Mean Square (LMS) algorithm. The LMS algorithm is widely used in adaptive filtering applications, where the goal is to minimize the mean square error (MSE) between the desired output and the actual system output by iteratively adjusting the filter coefficients. The coefficient update formula for LMS is given as in Eqn. (1)

$$w(n+1) = w(n) + \mu \cdot e(n) \cdot x(n) \quad (1)$$

Where:

- $w(n)$ is the current filter weight.
- μ is the step size.
- $e(n)$ is the error signal.
- $x(n)$ is the input signal.

However, the synchronous execution of error calculation and weight update leads to computational delays in hardware implementations, especially in high-speed real-time systems.

To address this issue, the paper introduces the Delayed Least Mean Square (D-LMS) algorithm. In this algorithm, a delay is introduced between the error calculation and weight update processes, allowing them to occur in different clock cycles. This decoupling reduces the system's critical path and improves convergence speed. The modified weight update equation for the D-LMS algorithm as in Eqn. (2)

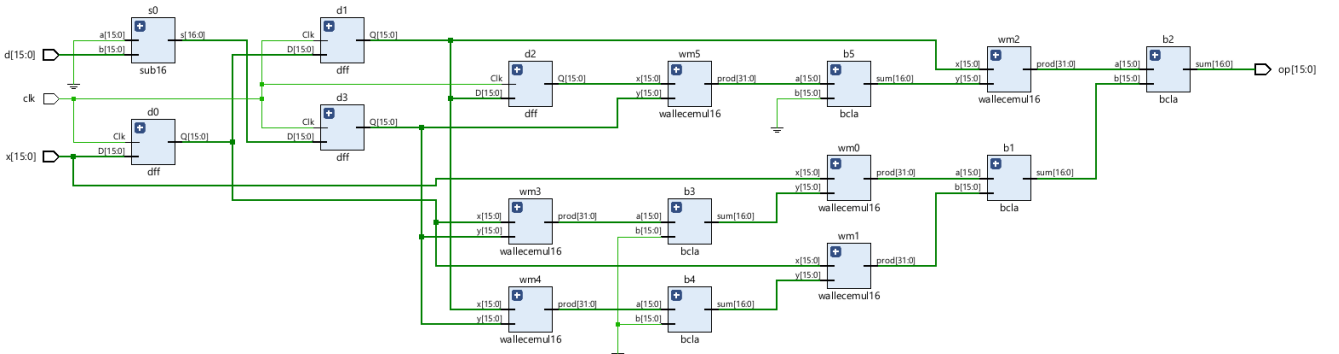


Figure 1 The RTL schematic of the D-LMS adaptive filter

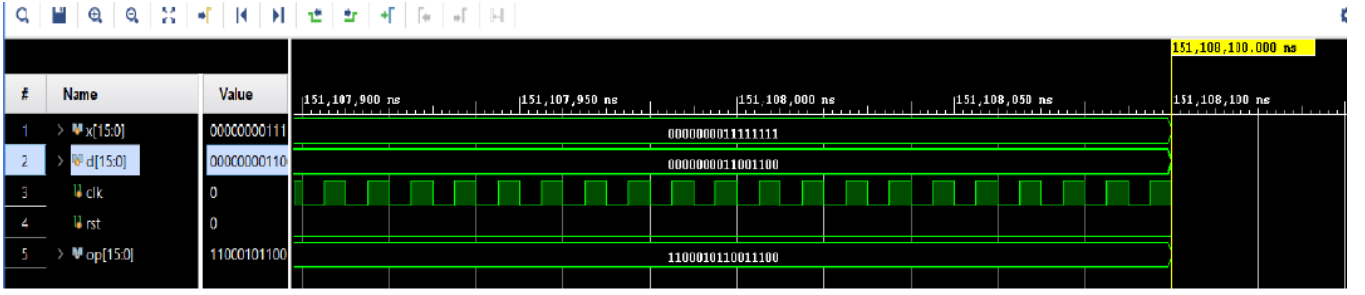


Figure 2 The output of Delayed LMS Adaptive filter

$$w(n+1) = w(n) + 2\mu \cdot e(n-m) \cdot x(n-m) \quad (2)$$

Where:

m represents the delay between the error signal and weight update. A delay of m is selected to achieve faster convergence while maintaining simplicity in implementation.

B. Algorithm Implementation

The next step in the process involves the hardware implementation of the D-LMS algorithm using Verilog Hardware Description Language (HDL). The focus is on designing efficient hardware components to optimize the performance of the adaptive filter. The important hardware sections are described below.

Block Carry Look-Ahead Adder: The D-LMS algorithm relies on fast arithmetic operations to update filter weights. A 16-bit Block Carry Look-Ahead Adder is used in the design, which allows faster addition operations compared to conventional ripple-carry adders. The look-ahead adder generates carry signals in parallel, reducing propagation delay and speeding up the weight update process.

Wallace Tree Multiplier: The D-LMS algorithm requires efficient multiplication operations. The paper employs a 16-bit Wallace Tree Multiplier, which uses carry-save adders to reduce the number of partial products generated during multiplication. This results in faster multiplication and minimizes the overall critical path. Additionally, Baugh-Wooley technique is used to handle signed multiplication.

To further optimize resource usage, multiplexers replace full adders within the Wallace Tree Multiplier. This change reduces the utilization of Look-Up Tables (LUTs) and power consumption while maintaining the same output accuracy as the original D-LMS algorithm. The modification aims to minimize the overhead in terms of hardware resources without compromising performance.

C. Modified D-LMS architecture

The Delayed Least Mean Square (D-LMS) algorithm is optimized for real-time applications through specific hardware setups. Both standard and modified D-LMS use Block Carry Look-Ahead Adders and Wallace Tree Multipliers. The modified D-LMS replaces full adders with multiplexers, reducing resource use and dynamic power consumption while maintaining accuracy. Its architecture also shortens critical paths, enhancing efficiency and lowering latency.

D. Simulation and Synthesis

After implementing the hardware design using Verilog HDL, the system is simulated and synthesized to verify its performance. The simulation is carried out using Xilinx Vivado Design Suite, a widely-used tool for designing and synthesizing digital circuits on FPGAs. Fig 1 shows the RTL schematic of the Delayed LMS adaptive filter.

E. Evaluation Metrics

The evaluation metrics include resource utilization (LUTs), power consumption, and simulation time. Resource utilization measures the number of LUTs used, which are essential in FPGA design as they represent the basic logic blocks. Power consumption assesses the dynamic energy usage of the D-LMS filter, ensuring it is efficient for real-time and embedded applications. Simulation time reflects the speed of execution and is vital for determining the design's feasibility in real-time systems.

F. FPGA Implementation

The final step in the process involves the implementation of the D-LMS adaptive filter on an FPGA platform. The design is synthesized and programmed onto a Nexys 4 DDR Artix-7 XC7A100T-CSG324-1 FPGA board. The FPGA provides a practical testing environment for evaluating the real-world performance of the adaptive filter.

IV. VALIDATION

A. FPGA Implementation Results

To validate the practical feasibility of the proposed design, a D-LMS adaptive filter was implemented on a Nexys 4 DDR Artix-7 FPGA board. Figure 2 presents the output of the FPGA-based test setup and the corresponding results. These results confirm the system's performance with real-world data.

Table 1 The area report of D-LMS adaptive filter

Resource	Utilization	Available	Utilization %
LUT	1278	134600	0.95%
FF	65	269200	0.02%
IO	49	400	12.25%

The results, presented in Table 1, demonstrate the area efficiency of the proposed design by showcasing its reduced logic resource usage compared to other alternatives.

Table 2 Delay report of D-LMS adaptive filter

Name	Slack	Levels	Routes	High Fan out	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	∞	15	15	90	d3/Q_reg[1]/C	op[14]	13.637	4.25	9.388	∞
Path 2	∞	15	15	74	d0/Q_reg[2]/C	op[15]	13.623	4.252	9.372	∞
Path 3	∞	14	14	90	d3/Q_reg[1]/C	op[13]	13.227	4.164	9.064	∞
Path 4	∞	14	14	90	d3/Q_reg[1]/C	op[12]	13.155	4.145	9.011	∞
Path 5	∞	14	14	90	d3/Q_reg[1]/C	op[11]	12.956	4.145	8.812	∞
Path 6	∞	13	13	90	d3/Q_reg[1]/C	op[9]	12.003	4.043	7.961	∞

The implementation results of the D-LMS adaptive filter design are evaluated based on resource utilization, power consumption, and simulation time.

An evaluation in terms of simulation time is done by measuring the processing speed, which is measured in nanoseconds. The delay introduced by the D-LMS filter is noted and presented in Table 2. These results reflect the overall system speed and its suitability for real-time applications.

Table 3 Comparison of power and resource utilization

PARAMETER	LMS algorithm based FIR adaptive filter	D-LMS algorithm based adaptive filter	Modified D-LMS algorithm based adaptive filter
Resource Utilisation (LUTs)	383	1301	1278
Dynamic Power	51.17mW	72.190mW	70.453mW
Simulation Time	10.987ns	8.690ns	8.690ns

A comparison in terms of resource utilization is reported in the Table 3. It presents the number of LUTs required for the D-LMS design and compares it with both traditional LMS and modified D-LMS designs. This provides a comprehensive comparison, summarizing the key improvements in resource utilization, power consumption, and convergence speed, which demonstrate the advantages of the proposed design. By replacing full adders with multiplexers in the multiplier block, the design significantly reduces power consumption, making the modified D-LMS filter more energy-efficient and enhancing both hardware efficiency and power optimization compared to existing LMS implementations.

V. RESULTS AND DISCUSSIONS

The proposed D-LMS adaptive filter offers several advantages, including improved convergence speed and reduced computational complexity, making it ideal for real-time applications. The simulation time for both standard and modified D-LMS algorithms is identical at 8.690 ns due to their similar operational structures, effective delay mechanisms, and unchanged computational workload despite optimizations in the modified version. FPGA implementation further balances timing through resource-based synthesis, ensuring that while modifications improve efficiency and resource usage, they do not affect execution speed in simulations.

The choice of step size μ is critical in adaptive filtering, as it directly impacts the algorithm's convergence speed and stability. Larger step sizes yield faster convergence but can lead to instability and increased steady-state error, while smaller step sizes enhance stability and accuracy at the expense of slower convergence.

By decoupling error computation from weight updates, the critical path is shortened, leading to quicker convergence and lower complexity. The use of optimized hardware components, such as Block Carry Look-Ahead Adders and Wallace tree Multipliers, enhances the system's resource utilization and power efficiency. Replacing full adders with multiplexers in the multiplier reduces power consumption without sacrificing performance, making the system suitable for low-power applications. Its flexibility, scalability, and practical FPGA-based testing demonstrate its real-world applicability across various fields, including noise cancellation and biomedical signal processing.

This work introduces several unique contributions, such as delay-based hardware optimization, where the decoupling of error calculation and weight update shortens the critical path for enhanced system efficiency. The optimization of the Wallace Tree Multiplier by replacing full adders with multiplexers reduces LUT usage, improving resource efficiency. Furthermore, a comparative performance analysis highlights the advantages of the proposed design over traditional LMS and D-LMS implementations, demonstrating the superior performance of the modified D-LMS in terms of power consumption, resource utilization, and convergence speed.

VI. CONCLUSION

The methodology employed in this paper is comprehensive, starting from the review of LMS and D-LMS algorithms, proceeding with hardware implementation and optimization, and culminating in simulation, synthesis, and FPGA implementation. By focusing on efficient hardware design and utilizing the flexibility of Verilog HDL, this study demonstrates how the D-LMS algorithm can be optimized for real-time, low-power, and high-speed applications. Overall, the results indicate that the modified D-LMS algorithm not only maintains execution speed but also significantly enhances performance metrics such as convergence speed and resource efficiency compared to traditional approaches. This positions the modified D-LMS as a highly effective solution for various practical applications in signal processing.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to the Head of the Department of Electronics & Communication Engineering, Government Engineering College, Thrissur, as well as the faculty and staff of the VLSI and Embedded Systems Laboratory for providing the laboratory facilities that enabled the successful completion of this work.

REFERENCES

- [1] B. Widrow and M.E. Hoff, "Adaptive Switching Circuits," IRE WESCON Conference Record, part 4, pp. 96–104, 1960.
- [2] J.H. Husøy and M.S.E. Abadi, "Unified approach to adaptive filters and their performance," IET Signal Process., vol. 2, no. 2, pp. 97–109, 2008.
- [3] C.R. Baugh and B.A. Wooley, "A Two's Complement Parallel Array Multiplication Algorithm," IEEE Trans. Comput., vol. C-22, no. 12, pp. 1045–1047, Dec. 1973.
- [4] X. Dong, H. Li, and Y. Wang, "High-speed FPGA implementation of an improved LMS algorithm," in Proc. IEEE Int. Conf. Field-Programmable Technol. (FPT), 2014, pp. 326–329.
- [5] P.K. Meher and S.Y. Park, "Critical-path analysis and low-complexity implementation of the LMS adaptive algorithm," IEEE Trans. Circuits Syst. I, vol. 61, no. 3, pp. 625–634, Mar. 2014.
- [6] P.K. Meher and S.Y. Park, "Area-delay power efficient fixed-point LMS adaptive filter with low adaptation delay," IEEE Trans. Circuits Syst. I, vol. 60, no. 1, pp. 146–156, Jan. 2013.
- [7] J. Li, M. Li, M. Meng, and Z. Liu, "The on-chip DLMS filter design method of wireless sensor node based on FPGA," Shock and Vibration, vol. 2020, Article ID 4850438, 16 pages, 2020.
- [8] Y. Yi, R. Woods, L.-K. Ting, and C.F.N. Cowan, "High-speed FPGA-based implementations of delayed-LMS filters," VLSI Signal Process. Syst., vol. 39, nos. 1–2, pp. 113–131, Aug. 2005.
- [9] J.G. Proakis and D.G. Manolakis, Digital Signal Processing, 4th ed. New Delhi, India: PHI, 2007.
- [10] S. Haykin, Adaptive Filter Theory, 5th ed. Upper Saddle River, NJ: Pearson Education, 2013.
- [11] B. Parhami, Computer Arithmetic: Algorithms and Hardware Designs, 2nd ed. New York, NY: Oxford Univ. Press, 2010.
- [12] K. Parhi, VLSI Digital Signal Processing Systems: Design and Implementation, New Delhi, India: Wiley India, 1999.
- [13] S. Brown and Z. Vranesic, Fundamentals of Digital Logic Design with Verilog, 2nd ed. New Delhi, India: Tata McGraw Hill, 2005.
- [14] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Upper Saddle River, NJ: Pearson Education, 2003.